

Phase 7 Assignment 7B - Backend API Projects

Goal: Master the entire course (Phase 1 through Phase 7) by building **distributed, production-grade backend systems** — multi-service platforms with event-driven communication, gRPC, real-time delivery, and AI integration. Scope: Only use topics taught in Lessons 1-59. Standard: The hardest assignment in the course. These are **enterprise, senior-level** systems — not applications, but **platforms** decomposed into services. Each is a portfolio centerpiece.

What "Phase 7" Means Here

Phase 7 is the **Expert / Distributed-Systems** tier (Lessons 56-59), built on the full Phase 1-6 foundation:

- **Microservices architecture** (Lesson 56) — service boundaries around business capabilities, database-per-service, an API gateway, inter-service communication, and resilience (timeouts, retries, graceful degradation).
- **Event-driven systems** (Lesson 57) — a **message broker** (RabbitMQ or Kafka) for pub/sub between services; the **Saga pattern** (orchestration or choreography) with **compensating actions**; **at-least-once** delivery with **idempotent** consumers; dead-letter handling.
- **gRPC** (Lesson 58) — Protocol Buffers + strongly-typed internal service-to-service calls (unary and streaming) where low latency and a strict contract matter.
- **LLM / AI patterns** (Lesson 59) — token-by-token streaming, per-user/per-tenant **token budgets**, conversation state, and cost control for AI features (a real provider or a simulated model is acceptable — the focus is the FastAPI patterns).

You are expected to combine these with **everything** from Phases 1-6: auth/RBAC/API keys, async databases with migrations and transactions, Redis, background tasks, WebSockets, SSE, caching, pagination/filtering/search, rate limiting, file uploads, testing, Docker, CI/CD, monitoring, versioning, and security.

Allowed Scope (Phase 1 - Phase 7)

Phase 1-2 (Fundamentals & Core): FastAPI routing, path/query/body params, Pydantic validation (`Field`, `field_validator`, `model_validator`, `model_config`), `response_model` + schema separation, status codes, `HTTPException`, custom exception handlers, `Depends()` (sub-deps, class-based, `yield`), middleware, `APIRouter`, form data + file uploads, headers + cookies.

Phase 3 (Databases): SQLAlchemy 2.0 / `SQLModel`, relationships (O2M, M2M, association objects, self-referential), the session as a dependency, `from_attributes`, Alembic migrations, **async SQLAlchemy**, database transactions, Redis, MongoDB (optional).

Phase 4 (Advanced): async reasoning + `run_in_threadpool`, auth (bcrypt, OAuth2, JWT access/refresh, RBAC, API keys), background tasks, WebSockets, SSE / streaming, CORS, rate limiting, caching, pagination (offset + cursor), filtering/sorting/searching, i18n (optional), GraphQL (optional).

Phase 5 (Testing): pytest, `TestClient` / `httpx.AsyncClient`, isolated test databases, `app.dependency_overrides`, fixtures, mocking, coverage + CI.

Phase 6 (Production): layered project structure, `pydantic-settings` config, structured logging + request-ID tracing, security best practices, performance (N+1, pooling, indexes), Docker + docker-compose, Gunicorn/Uvicorn workers, deployment, CI/CD pipelines, monitoring (health checks, metrics, Sentry), API versioning, OpenAPI customization.

Phase 7 (Expert / Distributed): microservices decomposition, event-driven communication via a message broker, the Saga pattern with compensation, gRPC internal APIs, and LLM/AI integration patterns.

Do not use (beyond Phase 7):

- Kubernetes / service mesh **as a hard requirement** (you may deploy on them as a stretch, but the assignment must run on `docker-compose`).
- Stream-processing frameworks (Kafka Streams, Flink, Spark), CQRS/event-sourcing **frameworks**, or a workflow engine (Temporal, Airflow) — model Sagas with the taught patterns (events + compensating actions).
- Real ML model training, blockchain, or any technique not taught in Lessons 1-59.

Global Rules

- Build **backend services only**. No frontend (a minimal HTML page to exercise a WebSocket/SSE stream is acceptable as a test harness).
- Do not provide solutions, code, pseudocode, or implementation hints in your submission notes.
- **Decompose each platform into multiple services** with clear boundaries. Each service owns its data — **no shared database** across services; services communicate only via **APIs, gRPC, or events**.
- **Persist all durable data in real databases** (PostgreSQL via SQLAlchemy 2.0 / SQLModel), each service with its own **Alembic** migrations. Use **async** database access.
- **Use a message broker** (RabbitMQ or Kafka) for cross-service events; consumers must be **idempotent** (dedupe by event id) and unprocessable messages must go to a **dead-letter queue**.
- **Use gRPC** for the specific internal, low-latency service-to-service calls each project names.
- **Implement real authentication and authorization**: hashed passwords, JWT access + refresh, RBAC, and API keys for machine/service callers.
- **Include real-time delivery** (WebSockets and/or SSE) where the project requires it, multi-worker-safe via **Redis Pub/Sub**.
- **Use background tasks** for after-response work (notifications, webhooks, digests, file processing).
- **Apply caching, rate limiting, pagination, and filtering** to the appropriate paths.
- **Containerize the whole stack** with `docker-compose` (services + Postgres instances + Redis + broker); provide a `Dockerfile` per service.
- **Provide a test suite** (unit + integration + API + RBAC) with an isolated test database and a coverage gate, plus a **CI/CD workflow**.
- **Instrument for production**: env-based config, structured logs, health/readiness endpoints, metrics, and secure headers.
- Each service must run with `uvicorn main:app` (or a Gunicorn/Uvicorn command) after `alembic upgrade head`, and the full platform must come up with one `docker-compose up`.

Recommended platform topology:

- An **API gateway** (a thin FastAPI service, or per-service public routers behind a reverse proxy) is the single public entry point; internal services are not publicly exposed.
- **One database per service** (separate Postgres schemas/instances). Cross-service reads happen via API/gRPC or via data carried on events — never by reaching into another service's tables.
- **Redis** for caching, rate-limit counters, and WebSocket/SSE fan-out.
- A **broker** (RabbitMQ/Kafka) for the event backbone; an **outbox** pattern (an `events` table written in the same transaction, published by a relay) is recommended for reliable publishing.

Recommended evaluation mindset:

- Correct **service boundaries** and strict **data ownership**.
- Correct **distributed workflows** — Sagas that compensate on failure; idempotent, at-least-once event handling.
- Correct **concurrency and integrity** — atomic money/stock/state operations; idempotency keys.
- Sound **real-time, caching, rate-limiting, and security** decisions.
- Professional **testing, containerization, CI/CD, and observability**.

Project 1 - LedgerCore: Payments & Double-Entry Ledger Platform

Difficulty Level

Expert

Estimated Completion Time

45-60 hours

Project Overview

Build the backend for **LedgerCore**, a payment-processing platform in the spirit of Stripe: merchants integrate via API keys, create payment intents, capture funds, issue refunds, and receive signed webhooks — while every movement of money is recorded in a strict **double-entry ledger** that always balances. The system is decomposed into services, communicates via events, and uses gRPC for the low-latency, must-be-atomic ledger posting. Correctness under concurrency and idempotency are the whole game.

Problem Statement

A fintech needs a backend that can:

- Let merchants authenticate as machines (API keys) and process payments programmatically.
- Model a payment's lifecycle (`requires_confirmation` → `authorized` → `captured` → `refunded` / `failed` / `cancelled`) with correct, controlled transitions.

- Record every financial event as **balanced double-entry** ledger transactions (debits equal credits), so balances are always derivable and auditable.
- Guarantee **idempotency** so a retried request never double-charges.
- Notify merchants of outcomes via **signed, retried webhooks**.
- Produce **reconciliation** and balance reports without ever trusting client-provided totals.

Functional Requirements

- Merchant onboarding and scoped **API-key** management (mint, rotate, revoke; keys hashed at rest).
- Create, confirm, capture, and cancel **payment intents**; issue full/partial **refunds**.
- Tokenized **payment methods** (never store raw card data — accept a token, model a reference only).
- **Double-entry ledger**: every charge/refund/fee/payout posts a balanced transaction across ledger accounts (merchant balance, platform fees, pending, payouts).
- **Idempotency keys** on all mutating money endpoints.
- **Webhooks**: register endpoints, deliver HMAC-signed events with retry/backoff and dead-lettering.
- **Payouts**: move a merchant's available balance out, posting the corresponding ledger entries.
- **Reconciliation & balance** reporting per merchant and platform-wide.
- **Disputes/chargebacks** (model the state machine and ledger impact).
- **Audit log** of every security- and money-sensitive action.

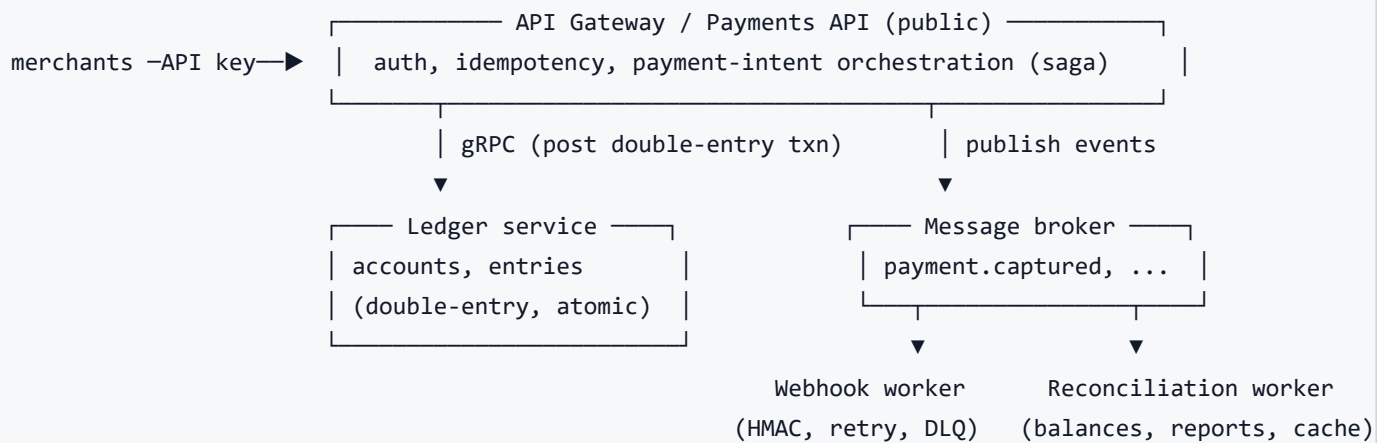
Non-Functional Requirements

- Financial correctness: the ledger must **always balance**; no operation may leave it inconsistent.
- Idempotent, at-least-once event handling; exactly-one ledger effect per financial event.
- Low-latency, atomic ledger posting via **gRPC** from the Payments service to the Ledger service.
- Resilience: broker/consumer failures must not lose events (outbox + retries + DLQ).
- Security: no raw card data; secrets in config; API keys hashed; HMAC-signed webhooks.

System Overview

LedgerCore is split into cooperating services around distinct capabilities: a public **Payments API**, an internal **Ledger** service (source of truth for balances), a **Webhook/Notification** worker, and a **Reconciliation** worker. The Payments service orchestrates each money flow as a **Saga** and posts to the Ledger over gRPC; financial events flow through the broker to the webhook and reconciliation consumers.

Suggested Architecture



- **Payments service** (public): API keys, payment intents, refunds, disputes, idempotency; orchestrates the Saga and calls the Ledger via gRPC.
- **Ledger service** (internal, gRPC): the only writer of ledger entries; enforces balanced double-entry atomically; owns balances.
- **Webhook worker** (event consumer): signs and delivers webhooks with retry/backoff → DLQ.
- **Reconciliation worker** (event consumer + scheduled): recomputes balances, produces reports (cached).

API Requirements

- Public endpoints authenticate with **API keys** (scoped: `payments:read`, `payments:write`, `payouts:write`, `webhooks:manage`); rate-limited **per merchant/plan**.
- All mutating money endpoints require an **Idempotency-Key** header; a repeated key returns the original result, never a second effect.
- The Payments service **never** writes ledger rows directly — it calls the Ledger service over **gRPC**.
- Financial state changes publish **events** to the broker; the Payments service uses an **outbox** so publishing is reliable.
- Amounts are integer **minor units** (e.g. cents) with an explicit currency; totals are server-computed.

Required API Endpoints

Method	Path	Auth	Purpose
POST	<code>/v1/merchants</code>	admin	Onboard a merchant
POST	<code>/v1/api-keys</code>	merchant/admin	Mint a scoped API key (shown once)
POST	<code>/v1/api-keys/{id}/rotate</code>	admin	Rotate a key
DELETE	<code>/v1/api-keys/{id}</code>	admin	Revoke a key
POST	<code>/v1/payment-methods</code>	API key	Register a tokenized method

Method	Path	Auth	Purpose
POST	/v1/payment-intents	API key	Create a payment intent (Idempotency-Key)
POST	/v1/payment-intents/{id}/confirm	API key	Authorize funds
POST	/v1/payment-intents/{id}/capture	API key	Capture (posts to ledger via gRPC)
POST	/v1/payment-intents/{id}/cancel	API key	Cancel / void
GET	/v1/payment-intents/{id}	API key	Retrieve one
GET	/v1/payment-intents	API key	List/filter (status, date, amount)
POST	/v1/refunds	API key	Full/partial refund (Idempotency-Key)
POST	/v1/disputes/{id}/accept / /challenge	API key	Dispute workflow
POST	/v1/payouts	API key	Pay out available balance
GET	/v1/balance	API key	Available + pending balance (from ledger)
POST	/v1/webhook-endpoints	API key	Register a webhook + secret
GET	/v1/reconciliation	admin	Reconciliation report (cached)
GET	/health/live · /health/ready · /metrics	internal	Ops

Request Expectations

- Payment-intent creation: **amount** (minor units, > 0), **currency**, **payment_method** (token), optional **capture_method** (automatic/manual), **metadata**; header **Idempotency-Key**.
- Refund: **charge_id**, optional **amount** (≤ remaining), **reason**.
- Payout: **amount** (≤ available balance), **currency**.
- Webhook endpoint: **url**, **enabled_events**.

Response Expectations

- Payment intent: **id**, **status**, **amount**, **currency**, **charge_id** (when captured), timestamps; **never** any raw card data.
- Balance: **available**, **pending**, **currency** — derived from ledger entries, not stored mutable fields.
- Webhook payloads: an envelope { **id**, **type**, **created**, **data** }, HMAC-signed (signature header).
- Errors: a consistent envelope with **error_code**, **detail**, **request_id**.

Validation Requirements

- `amount/minor units`: positive integers; refunds $\leq$ remaining capturable/refundable; payouts $\leq$ available balance.
- `currency`: 3-letter uppercase; consistent within a payment.
- API-key scopes enforced per endpoint.
- `Idempotency-Key`: required on money mutations; validated format; stored with the request fingerprint.
- Status transitions constrained to the defined lifecycle.

Business Rules

- **Double-entry invariant**: every posted transaction's debits equal its credits; the ledger balances at all times.
- **Idempotency**: a repeated `Idempotency-Key` returns the stored original response and causes **no** second money movement.
- **Exactly-one ledger effect** per financial event, even under at-least-once event delivery (idempotent consumers).
- Only an `authorized` intent can be captured; only a `captured` charge can be refunded; a `refunded` charge cannot be captured again.
- Capture is a **Saga**: reserve $\rightarrow$ post to ledger (gRPC) $\rightarrow$ mark captured $\rightarrow$ publish `payment.captured`; on ledger failure, **compensate** (void authorization) and mark failed.
- A payout may not exceed available balance and posts matching ledger entries.
- Webhooks are delivered **at least once**, HMAC-signed, retried with backoff, and dead-lettered after N attempts.
- No endpoint trusts a client-provided balance or total — all are server-derived from the ledger.

Edge Cases

- Concurrent captures of the same intent must result in exactly one capture (idempotency/locking).
- A retried create with the same `Idempotency-Key` but a different body returns a `409` (fingerprint mismatch).
- Ledger service unreachable mid-capture $\rightarrow$ Saga compensates; the intent does not end up captured-without-ledger.
- Refund exceeding remaining amount $\rightarrow$ `409`.
- Payout exceeding available balance $\rightarrow$ `409`.
- A webhook endpoint that 500s repeatedly $\rightarrow$ deliveries go to the DLQ; the platform is unaffected.
- Duplicate broker delivery of `payment.captured` $\rightarrow$ the consumer applies it once.

Suggested Database Schema

Payments service DB - `merchants` (id, name, status, plan, created_at) - `api_keys` (id, merchant_id, name, key_hash, prefix, scopes, last_used_at, revoked_at) - `payment_methods` (id, merchant_id, token, brand, last4, created_at) — reference only, no PAN - `payment_intents` (id, merchant_id, amount, currency, status, capture_method, payment_method_id, metadata, created_at) - `charges` (id, intent_id, amount_captured,

amount_refunded, status, created_at) - **refunds** (id, charge_id, amount, reason, status, created_at) - **disputes** (id, charge_id, status, amount, created_at) - **idempotency_keys** (key, merchant_id, request_fingerprint, response_snapshot, created_at, **unique(key, merchant_id)**) - **outbox_events** (id, aggregate_id, type, payload, published_at) — reliable publishing - **webhook_endpoints** (id, merchant_id, url, secret, enabled_events, is_active) - **webhook_deliveries** (id, endpoint_id, event_id, status, attempts, next_retry_at) - **audit_log** (id, merchant_id, actor, action, target, created_at)

Ledger service DB (separate) - **ledger_accounts** (id, merchant_id nullable, type [merchant_balance/pending/fees/payouts], currency) - **ledger_transactions** (id, external_ref, description, created_at) - **ledger_entries** (id, transaction_id, account_id, direction [debit/credit], amount, **balanced per transaction**) - **payouts** (id, merchant_id, amount, currency, status, created_at)

Expected Folder Structure

```
ledgercore/
  gateway/                # or payments service exposes the public API directly
  services/
    payments/
      app/ (main.py, api/, services/, repositories/, models/, schemas/, core/, db/)
      proto/ (ledger.proto)    # gRPC client stubs
      alembic/
      tests/
      Dockerfile
    ledger/
      app/ ...                # gRPC server + REST for internal ops
      proto/ (ledger.proto)
      alembic/ tests/ Dockerfile
    webhook_worker/          # event consumer
    reconciliation_worker/
  docker-compose.yml        # services + postgres(x2) + redis + rabbitmq/kafka
  .github/workflows/ci-cd.yml
  proto/ (shared .proto)
  README.md
```

Deliverables

- A multi-service, runnable platform (**docker-compose up**) with Payments (public), Ledger (gRPC), and event-consuming workers.
- Alembic migrations per service; the double-entry ledger schema.
- A gRPC contract (**.proto**) and working Payments→Ledger calls.
- Idempotency, outbox-based event publishing, signed/retried webhooks with a DLQ.
- Seed data (merchants, API keys, sample intents/charges/refunds, ledger accounts/entries).
- A test suite proving **financial correctness** (ledger always balances; idempotent captures; Saga compensation), plus RBAC/API-key tests.
- README covering the architecture, the payment lifecycle, the Saga, and the gRPC/event contracts.

Evaluation Criteria

- Correct, always-balanced double-entry ledger and server-derived balances.
- Correct idempotency (no double-charge) and exactly-one ledger effect under at-least-once delivery.
- Correct capture Saga with compensation on ledger failure.
- Clean service boundaries; gRPC used appropriately for the ledger posting.
- Reliable event publishing (outbox) and delivery (retries + DLQ); signed webhooks.
- Strong testing of financial invariants; production hygiene (config, logging, health, security).

Bonus Challenges

- Add **multi-currency** payouts with FX ledger entries.
- Add a **scheduled** reconciliation job that flags any ledger drift and alerts.
- Add **partial captures** and multi-capture flows with correct ledger effects.
- Add a **GraphQL** read API for merchant financial reporting (Lesson 39).

Project 2 - Fulfillment: Multi-Vendor Marketplace Order-Orchestration Platform

Difficulty Level

Senior-Level

Estimated Completion Time

50-65 hours

Project Overview

Build **Fulfillment**, the backend for a customer-facing multi-vendor marketplace where a single order can span **multiple vendors** and must be orchestrated across **inventory reservation**, **payment**, and **shipment** — using the **Saga pattern** with compensating actions when any step fails. Customers browse and search a catalog, place orders, and track fulfillment in **real time**; vendors manage products and fulfill their portion of orders. This is the microservices-decomposition and distributed-workflow showcase.

Problem Statement

A marketplace needs a backend that can:

- Serve a searchable, filterable, paginated **catalog** across many vendors.

- Accept an order that contains items from **several vendors**, and orchestrate its fulfillment as one coherent workflow.
- **Reserve inventory** per vendor, **capture payment**, and **create shipments** — and **compensate** (release stock, refund) if any step fails, leaving no partial mess.
- Let customers **track** their order live.
- Let vendors manage products, stock, and fulfill their share of orders.
- Isolate services so each owns its data and they cooperate via **events** and **gRPC**.

Functional Requirements

- **Catalog**: vendors, products, variants, categories, images (uploads), pricing; search/filter/sort/pagination.
- **Inventory**: stock per variant per vendor; **reserve/release** operations (strongly consistent, gRPC).
- **Cart** → **Checkout** → **Order**: build an order spanning multiple vendors; server-computed totals.
- **Order orchestration (Saga)**: on `order.placed` → reserve inventory (per vendor) → capture payment → create per-vendor shipments → confirm; on any failure, run compensations (release reservations, refund).
- **Payment** (mock service): authorize/capture/refund via events/API.
- **Shipping**: create shipments per vendor, track status transitions.
- **Real-time tracking**: customers watch order/shipment status via SSE/WebSockets.
- **Vendor operations**: manage products/stock; view and fulfill their portion of orders.
- **Notifications**: order confirmations, shipment updates (background + events).
- **Admin**: platform dashboards, vendor management, dispute handling.

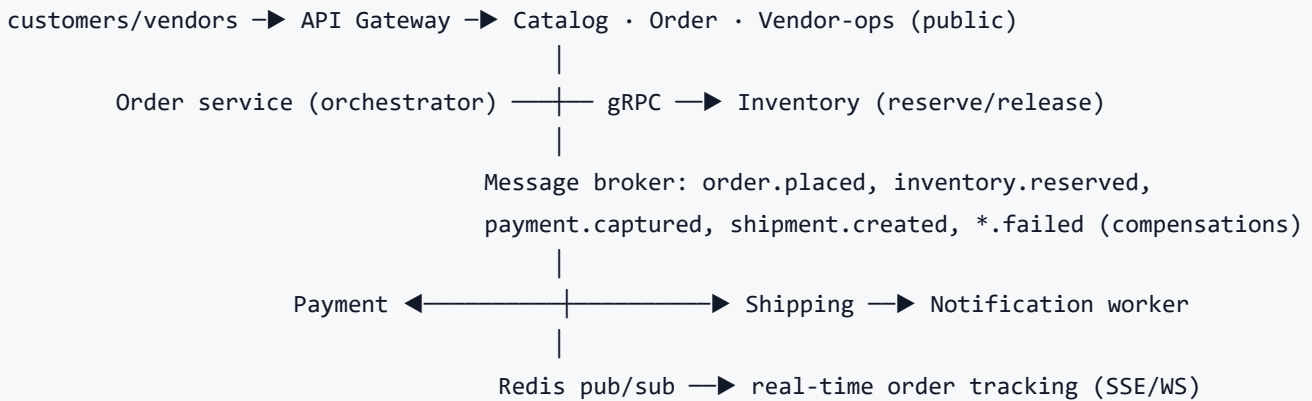
Non-Functional Requirements

- Distributed correctness: an order is either fully placed or fully compensated — **no partial orders**.
- Inventory integrity: no overselling under concurrency; reservations are atomic.
- Strong service isolation: catalog, inventory, order, payment, shipping each own their data.
- gRPC for the order-orchestrator → inventory reserve/release (must be fast and consistent).
- Scalable reads: hot catalog cached; large lists paginated.

System Overview

The platform is decomposed into **Catalog**, **Inventory**, **Order** (the orchestrator), **Payment**, **Shipping**, and **Notification** services behind an **API gateway**. The Order service runs the fulfillment **Saga**, calling Inventory over **gRPC** and coordinating Payment and Shipping via **events**. Customers connect to a real-time channel for live order status.

Suggested Architecture



- **Catalog service** (public): products/variants/categories/images; search/filter/sort/pagination; cached.
- **Inventory service** (gRPC + events): stock and atomic reserve/release; the orchestrator calls it over gRPC.
- **Order service** (orchestrator): builds orders, runs the **Saga**, tracks saga state, emits/consumes events.
- **Payment service** (mock): authorize/capture/refund.
- **Shipping service**: per-vendor shipments and tracking.
- **Notification worker**: emails/updates on order/shipment events; pushes real-time updates via Redis.

API Requirements

- Public endpoints use JWT (customers/vendors) with **RBAC** (**customer**, **vendor**, **admin**); catalog reads may be public.
- The Order orchestrator calls **Inventory over gRPC** for reserve/release; it coordinates Payment and Shipping via **events**.
- The fulfillment workflow is a **Saga** with a persisted saga state and **compensating actions** on failure.
- Catalog list endpoints support **search, filter, sort, and cursor pagination**; hot catalog data is **cached** with invalidation on vendor updates.
- Customers subscribe to a **real-time** order-status stream.

Required API Endpoints

Method	Path	Auth	Purpose
GET	/catalog/products	public	Search/filter/sort/paginate products
GET	/catalog/products/{id}	public	Product + variants + stock summary
POST	/vendor/products	vendor	Create a product (+ images)
PATCH	/vendor/products/{id}	vendor	Update product/pricing
POST	/vendor/inventory/{variant_id}	vendor	Adjust stock

Method	Path	Auth	Purpose
POST	<code>/cart</code> · <code>/cart/items</code>	customer	Build a cart
POST	<code>/checkout</code>	customer	Place an order (starts the Saga)
GET	<code>/orders/{id}</code>	customer/vendor	Order + line items + fulfillment state
GET	<code>/orders</code>	customer	List the caller's orders (cursor)
GET	<code>/orders/{id}/track</code>	customer	SSE live status stream
GET	<code>/vendor/orders</code>	vendor	The vendor's portion of orders
POST	<code>/vendor/orders/{id}/fulfill</code>	vendor	Mark items shipped
POST	<code>/orders/{id}/cancel</code>	customer	Cancel when allowed (triggers compensation)
GET	<code>/admin/dashboard</code>	admin	Aggregate metrics (cached)
WS	<code>/ws/orders</code>	customer	Real-time order/shipment events
GET	<code>/health/live</code> · <code>/health/ready</code> · <code>/metrics</code>	internal	Ops

Request Expectations

- Checkout: cart reference or `items` (each: `variant_id`, `quantity`), shipping address, payment token.
- Vendor product: `name`, `description`, `category_id`, `variants` (sku, price, attributes), images (multipart).
- Inventory adjust: `delta` or absolute `quantity`.

Response Expectations

- Order: `id`, `status` (`placed`/`confirmed`/`partially_shipped`/`completed`/`cancelled`/`failed`), per-vendor sub-orders, server-computed totals, saga progress.
- Product list: `items`, `next_cursor`, `has_more`, `filters_applied`.
- Real-time events: typed envelope (`order.confirmed`, `shipment.created`, `order.failed`, ...).

Validation Requirements

- `quantity`: positive; not exceeding available stock at reservation time (else compensation).
- Prices/totals: server-computed from catalog; never trusted from the client.
- Variant/category ids must reference existing rows; SKUs unique per vendor.
- Sort fields whitelisted; pagination `limit` bounded.
- Address and payment token validated before the Saga starts.

Business Rules

- An order that spans multiple vendors is one workflow: **all steps succeed or the order is fully compensated** (reservations released, payment refunded).
- **Inventory reservation is atomic** — no overselling under concurrent checkouts.
- The Saga persists its state; each step is **idempotent**, and every event is applied once.
- Payment is captured only after all reservations succeed; shipments are created only after payment.
- Cancelling before shipment triggers compensation (release + refund); after shipment, cancellation is disallowed or becomes a return flow.
- Vendors see and act only on their portion of an order (RBAC + scoping).
- Catalog updates invalidate the relevant cached entries.

Edge Cases

- Two customers check out the last unit concurrently → exactly one reservation succeeds; the other order compensates and fails cleanly.
- Inventory service returns success but payment fails → reservations are released (compensation).
- Duplicate `order.placed` delivery → the Saga runs once.
- A vendor deletes a product mid-order → the in-flight order is handled per policy (reservation already held).
- Partial fulfillment: one vendor ships, another cannot → order becomes `partially_shipped`; compensation only for the failed portion.
- Real-time stream reconnect → the client resyncs current order state.

Suggested Database Schema

Catalog service: `vendors`, `categories`, `products` (vendor_id, name, category_id, status), `product_variants` (product_id, sku unique-per-vendor, price, attributes), `product_images` (product_id, storage_key). **Inventory service:** `stock` (variant_id, vendor_id, available, reserved), `reservations` (id, order_id, variant_id, quantity, status). **Order service:** `orders` (id, customer_id, status, total, created_at), `order_items` (order_id, vendor_id, variant_id, quantity, unit_price), `sub_orders` (order_id, vendor_id, status), `order_sagas` (order_id, step, status, compensation_state), `order_events` (outbox). **Payment service:** `payments` (order_id, amount, status). **Shipping service:** `shipments` (id, sub_order_id, vendor_id, status, tracking_number), `shipment_events`. **Users:** `customers`, `vendor_users`, `admins`.

Relationships: vendor→products→variants (O2M); order→items→(variant, vendor); order→sub_orders (per vendor); reservation↔order; shipment↔sub_order. Junctions/association objects for order items and saga steps.

Expected Folder Structure

```
fulfillment/  
  gateway/  
  services/  
    catalog/      (app/, alembic/, tests/, Dockerfile)  
    inventory/    (app/, proto/inventory.proto, alembic/, tests/, Dockerfile)  
    order/        (app/, proto/, services/saga.py, alembic/, tests/, Dockerfile)  
    payment/      shipping/      notification_worker/  
docker-compose.yml # services + postgres(per svc) + redis + broker  
.github/workflows/ci-cd.yml  
proto/  
README.md
```

Deliverables

- A multi-service marketplace runnable via `docker-compose up`.
- A working **fulfillment Saga** with compensation, demonstrated end-to-end (success + failure paths).
- gRPC contract + Order→Inventory reserve/release calls; the event backbone across services.
- Real-time order tracking (SSE/WS) and searchable/paginated catalog with caching.
- Alembic migrations per service; seed data (vendors, products/variants, stock, sample orders across statuses).
- Tests proving no overselling, correct compensation, and idempotent event handling; RBAC tests.
- README covering the service map, the Saga, and the gRPC/event contracts.

Evaluation Criteria

- Correct multi-vendor **Saga** with compensation (no partial orders; no overselling).
- Clean service boundaries and data ownership; gRPC used for inventory reservation.
- Idempotent, at-least-once event handling; reliable publishing.
- Effective catalog search/filter/sort/pagination and caching; correct real-time tracking.
- Solid testing, containerization, CI/CD, and observability.

Bonus Challenges

- Add a **returns/RMA** flow with its own Saga (return authorized → item received → refund posted).
- Add **stock-reservation expiry** (a reservation auto-releases if the Saga stalls).
- Add a **search microservice** with proper full-text search over the catalog.
- Add **per-vendor rate limits** and vendor **API keys** for programmatic catalog updates.

Project 3 - VitalLink: Telehealth Virtual Care Platform (Capstone-Level)

Difficulty Level

Capstone-Level

Estimated Completion Time

60-80 hours

Project Overview

Build **VitalLink**, a privacy-conscious telehealth platform connecting patients and providers for virtual care. It combines **everything** in the course: multi-role identity, provider search and scheduling, an **AI symptom-triage** assistant (LLM, streamed), **real-time virtual visits**, clinical notes with **AI summarization**, e-prescriptions, and insurance-claim generation — decomposed into services and orchestrated via events. This is the AI-plus-real-time-plus-distributed integration finale, handling sensitive data with production-grade security.

Problem Statement

A telehealth company needs a backend that can:

- Register and verify **patients** and **providers** (with specialties/licenses), enforcing strict role-based access to sensitive health data.
- Let patients **search providers** and **book appointments** against real availability.
- Offer an **AI symptom-triage** assistant that streams guidance token-by-token, respects **per-patient token budgets**, and only reasons over data the user is permitted to see.
- Run **virtual visits** in real time (waiting room, presence, visit signaling).
- Capture **clinical notes** (with optional **AI summarization** of the visit) and issue **e-prescriptions**.
- Generate **insurance claims** from completed visits and track their status.
- Decompose the platform into services communicating via **events** and **gRPC**, with full production hardening and an audit trail over all PHI access.

Functional Requirements

- **Identity & verification**: patients, providers (specialty, license, verification status), staff; JWT + refresh; RBAC.
- **Provider directory & scheduling**: searchable directory; provider **availability**; **appointment booking** against slots (no double-booking).
- **AI triage** (LLM): a streamed, conversational symptom-triage assistant; per-patient/plan **token budgets**; persisted conversation; permission-filtered context; usage metering.

- **Virtual visits:** waiting room (SSE), real-time presence and visit signaling (WebSockets), visit lifecycle (`scheduled` → `in_progress` → `completed`).
- **Clinical:** clinical notes per visit; optional **AI note summarization** (streamed); e-prescriptions with medication references.
- **Billing/Claims:** generate a claim from a completed visit; claim line items; status workflow.
- **Notifications:** appointment reminders, visit-ready alerts, prescription notices (background + events; optional i18n).
- **Audit:** append-only log of every access to or change of patient health data.

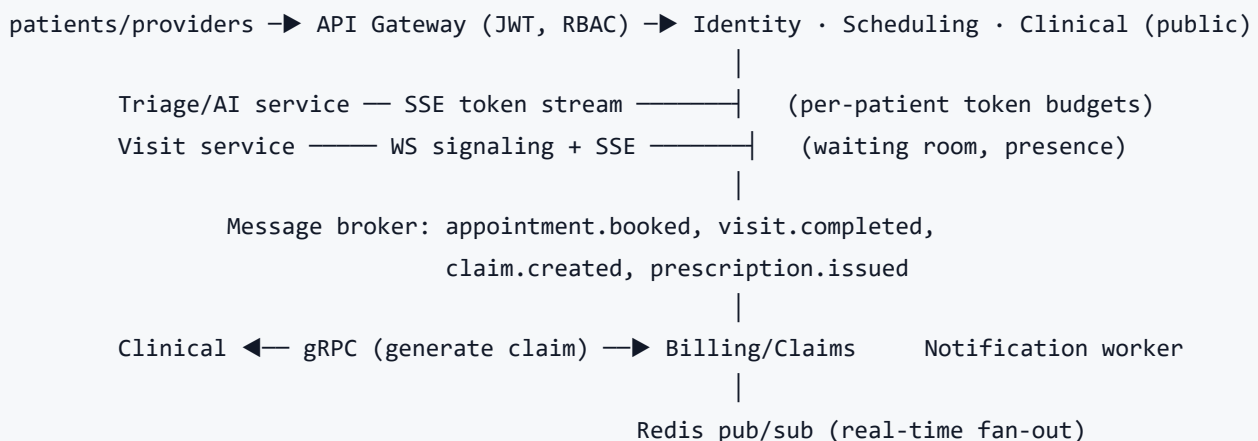
Non-Functional Requirements

- **Security/privacy first:** strict RBAC and tenant/patient-scoping; PHI-style fields never leak in responses or logs; secrets in config; audit every sensitive access.
- **AI cost & safety:** token budgets enforced; the assistant only sees permitted data; usage metered; graceful degradation if the AI service is down.
- **Real-time reliability:** authenticated WS/SSE; multi-worker fan-out via Redis.
- **Distributed correctness:** event-driven workflows (visit → note → claim) are idempotent; billing operations are transactional.
- Full production posture: config, structured logs, health/metrics, Docker, CI/CD, versioned API, customized OpenAPI.

System Overview

VitalLink is decomposed into **Identity**, **Scheduling**, **Triage/AI**, **Visit**, **Clinical**, **Billing/Claims**, and **Notification** services behind an **API gateway**. The **AI service** streams triage and note-summary tokens (SSE); the **Visit service** carries real-time signaling (WebSockets); events tie the workflow together (booking → reminders; visit-completed → note draft + claim); **gRPC** connects Clinical and Billing for claim generation.

Suggested Architecture



- **Identity service:** users/patients/providers, verification, auth, RBAC.
- **Scheduling service:** availability + appointments; no double-booking (atomic slot claim).

- **Triage/AI service:** streamed LLM triage; token budgets; conversation store; usage meter.
- **Visit service:** virtual-visit lifecycle; real-time waiting room (SSE) + signaling/presence (WS).
- **Clinical service:** clinical notes (+ AI summary), prescriptions; calls Billing over gRPC.
- **Billing/Claims service:** claims + line items; status workflow; transactional.
- **Notification worker:** reminders/alerts (background + events; optional i18n).

API Requirements

- JWT auth with **RBAC** roles: `patient`, `provider`, `nurse`, `billing`, `admin`; every PHI access checked and audited.
- The **AI triage** and **note-summary** endpoints **stream tokens over SSE** and enforce **per-patient/plan token budgets** (429 on exhaustion).
- **Virtual visit** signaling and presence run over **WebSockets**; the waiting room streams via **SSE**; both authenticated before accept.
- Completed visits publish events that (a) draft a clinical note via the AI service and (b) trigger claim generation (Clinical → Billing over **gRPC**).
- Sensitive fields are never returned to unauthorized roles and never logged.

Required API Endpoints

Method	Path	Auth	Purpose
POST	<code>/auth/register</code> · <code>/auth/login</code> · <code>/auth/refresh</code>	public/user	Auth
POST	<code>/providers/{id}/verify</code>	admin	Verify a provider's license
GET	<code>/providers</code>	user	Search directory (specialty, availability)
GET	<code>/providers/{id}/availability</code>	user	Open slots
POST	<code>/appointments</code>	patient	Book a slot (atomic; no double-book)
POST	<code>/appointments/{id}/cancel</code>	patient/provider	Cancel per policy
POST	<code>/triage/sessions</code>	patient	Start an AI triage session
GET	<code>/triage/sessions/{id}/stream</code>	patient	SSE streamed triage (token budget)
GET	<code>/triage/usage</code>	patient/admin	Token usage vs budget
POST	<code>/visits/{id}/start</code> · <code>/join</code> · <code>/end</code>	provider/patient	Visit lifecycle
GET	<code>/visits/{id}/waiting-room</code>	patient	SSE waiting-room stream
WS	<code>/ws/visits/{id}</code>	participant	Real-time signaling + presence

Method	Path	Auth	Purpose
POST	/visits/{id}/notes	provider	Clinical note
GET	/visits/{id}/notes/summarize	provider	SSE AI note summary
POST	/prescriptions	provider	Issue an e-prescription
GET	/claims · POST /claims/{id}/submit	billing	Claims workflow
GET	/audit-log	admin	PHI-access audit (paginated)
GET	/health/live · /health/ready · /metrics	internal	Ops

Request Expectations

- Appointment: `provider_id`, `slot_id` (or `starts_at`), `reason`.
- Triage session: initial `symptoms`/message; subsequent messages continue the conversation.
- Clinical note: `visit_id`, structured fields (subjective/objective/assessment/plan).
- Prescription: `patient_id`, `medication_id`, `dosage`, `duration`.

Response Expectations

- Provider directory: paginated, filtered results with `specialty`, `rating`, availability summary — no private contact data beyond what a booking needs.
- Triage stream: SSE token events + a completion event with `input_tokens/output_tokens` used.
- Visit/waiting-room events: typed envelopes (`participant.joined`, `visit.started`, ...).
- Patient/provider responses hide sensitive fields per the caller's role.

Validation Requirements

- Booking: the slot must be open and in the future; a patient cannot double-book overlapping slots; a provider's slot cannot be double-claimed (atomic).
- Triage: `symptoms`/message length bounds; token budget checked before streaming.
- Prescription: medication id exists; dosage/duration within allowed patterns; only a verified provider may prescribe.
- Claim amounts: server-derived from visit + plan; never client-trusted.
- Role/permission enforced on every PHI endpoint.

Business Rules

- **Strict access control:** a patient sees only their own records; a provider sees only their patients' records for active/authorized visits; cross-patient access returns `404`.
- **No double-booking:** claiming a slot is atomic; concurrent bookings resolve to exactly one.

- **AI budgets:** triage/summary streaming stops with **429** when the patient's token budget is exhausted; usage is metered per request.
- **AI safety:** the assistant only receives context the requesting user is permitted to see; the AI service being down degrades gracefully (feature disabled, not a 500 storm).
- **Event-driven clinical→billing:** **visit.completed** → draft note (AI) + generate claim (Clinical→Billing over gRPC); handlers are idempotent.
- **Auditability:** every read/write of PHI is recorded in an append-only audit log.
- **Prescriptions** may be issued only by verified providers for patients with a completed/active visit.

Edge Cases

- Two patients booking the same slot concurrently → one succeeds, one gets **409**.
- Triage stream exhausts the token budget mid-response → the stream ends cleanly with a budget-exceeded signal.
- AI service unreachable → triage/summary return a graceful "temporarily unavailable," not a 500; the rest of the platform works.
- A provider tries to access a patient they have no visit with → **404** (no existence leak) + audit entry.
- Duplicate **visit.completed** event → note/claim generated once.
- WebSocket visit reconnect → participant presence resyncs.
- Cancelling an in-progress visit vs a scheduled one → different allowed transitions.

Suggested Database Schema

Identity: **users**, **patients** (user_id, dob, ...), **providers** (user_id, specialty, license_no, verification_status), **roles**. **Scheduling:** **availability_slots** (provider_id, starts_at, ends_at, status), **appointments** (id, patient_id, provider_id, slot_id, status, reason). **Triage/AI:** **triage_sessions** (id, patient_id, created_at), **triage_messages** (session_id, role, content, input_tokens, output_tokens), **ai_usage** (patient_id, period, tokens_used). **Visit:** **visits** (id, appointment_id, status, started_at, ended_at), **visit_participants** (visit_id, user_id, role, joined_at). **Clinical:** **clinical_notes** (visit_id, subjective, objective, assessment, plan, ai_summary, author_id), **prescriptions** (id, patient_id, provider_id, medication_id, dosage, duration, issued_at), **medications**. **Billing:** **insurance_policies** (patient_id, payer, member_id), **claims** (id, visit_id, patient_id, status, total), **claim_items** (claim_id, code, amount). **Cross-cutting:** **notifications**, **consents** (patient_id, type, granted_at), **audit_log** (actor_id, patient_id, action, target, created_at).

Relationships: provider→availability→appointments→visits→notes/claims (chained O2M); patient→appointments/prescriptions/claims; visit↔participants (M2M via association object with a role); claim→claim_items (O2M). Self-referential/threaded where useful (triage messages ordered per session).

Expected Folder Structure

```
vitallink/  
  gateway/  
  services/  
    identity/    scheduling/    triage_ai/    visit/    clinical/    billing/  
    notification_worker/  
  (each service): app/ (main.py, api/, services/, repositories/, models/, schemas/, core/, db/, re  
                    proto/ (where gRPC used), alembic/, tests/, Dockerfile  
  docker-compose.yml    # services + postgres(per svc) + redis + broker  
  .github/workflows/ci-cd.yml  
  proto/  
  README.md
```

Deliverables

- A multi-service telehealth platform runnable via `docker-compose up`.
- Working **AI triage + note summarization** (streamed, token-budgeted), **real-time visits** (WS/SSE), and the **event-driven visit→note→claim** workflow (Clinical→Billing over gRPC).
- Strict RBAC + PHI scoping + an **audit log** over all sensitive access.
- Alembic migrations per service; seed data (patients, verified providers, availability, sample appointments/visits/notes/claims).
- Tests: RBAC/PHI-isolation, no-double-booking, token-budget enforcement, idempotent event handling, and AI-down graceful degradation.
- README covering the service map, the AI patterns, the real-time protocols, the event/gRPC contracts, and the security model.

Evaluation Criteria

- Airtight RBAC and PHI isolation; complete audit trail; no sensitive-data leakage.
- Correct AI streaming with enforced token budgets and graceful degradation.
- Correct real-time visit signaling and atomic, no-double-booking scheduling.
- Correct event-driven, idempotent clinical→billing workflow (gRPC + events).
- Clean service boundaries and full production hardening (config, logging, health/metrics, Docker, CI/CD, versioned + documented API).

Bonus Challenges

- Add **i18n** for patient notifications negotiated from `Accept-Language` (Lesson 38).
- Add **prompt/context caching or retrieval** so the triage assistant is grounded in the patient's own (permitted) history.
- Add a **GraphQL** reporting surface for admin analytics (Lesson 39).

- Add **consent-gated data sharing** so a provider can access records only after explicit patient consent, enforced and audited.

Final Submission Checklist

For each project, submit:

- The full multi-service repository (one folder per service; a gateway; shared `proto/`).
- Per-service `app/` (routers, services, repositories, models, schemas, core, db, realtime where used).
- `proto/` gRPC contracts and generated stubs where gRPC is used.
- Per-service `alembic/` migrations and seed data.
- Event definitions and consumer code (idempotent; with DLQ handling).
- `docker-compose.yml` bringing up all services + databases + Redis + the broker.
- Per-service `Dockerfile`; a CI/CD workflow.
- A test suite (unit + integration + API + RBAC) with a coverage gate.
- A README with the architecture diagram, service map, workflow/Saga descriptions, and the event/gRPC contracts.

Before submitting, verify:

- `docker-compose up` brings the **entire platform** online; each service migrates and starts.
- Each service owns its own database; no service reads another's tables directly.
- Cross-service calls use **APIs, gRPC, or events** — and the named gRPC calls actually use gRPC.
- Distributed workflows are **Sagas with compensation**; no partial/inconsistent end states.
- Event consumers are **idempotent** and **at-least-once-safe**; unprocessable messages dead-letter.
- Auth, RBAC/API-key scopes, tenant/patient isolation, rate limits, and idempotency all behave as specified.
- Real-time channels authenticate before accepting and fan out correctly across workers (Redis).
- AI features stream, enforce token budgets, and degrade gracefully.
- Money/stock/state operations are atomic and correct under concurrency.
- Tests pass in CI with the coverage gate; no secrets in code, logs, or responses.
- No concept beyond Phase 7 (Lessons 1-59) was required.

Difficulty Progression

Project	Difficulty	Main Focus
LedgerCore — Payments & Double-Entry Ledger	Expert	Idempotency, double-entry integrity, capture Saga, gRPC ledger, signed webhooks
Fulfillment — Multi-Vendor Marketplace Orchestration	Senior-Level	Microservices decomposition, fulfillment Saga + compensation, gRPC inventory, real-time tracking
VitalLink — Telehealth Virtual Care Platform	Capstone-Level	AI streaming + budgets, real-time visits, event-driven clinical→billing, PHI security, full integration

Complete the projects in order. Each assumes fluency with everything in the course. Together they demonstrate that you can architect, build, secure, test, containerize, and operate **distributed, real-time, AI-integrated backend platforms** — the mark of a senior backend engineer, and the true culmination of Phases 1 through 7.